

```
In [26]: from tensorflow.keras import datasets, layers, models
```

```
In [27]: import tensorflow as tf
```

```
In [28]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [29]: (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
```

```
In [30]: x_train, x_test = x_train /255.0, x_test /255.0

class_name = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'h
```

```
In [31]: print(y_train.shape)
print(y_train[0])
```

```
(50000, 1)
[6]
```

```
In [32]: plt.figure(figsize=(10, 2))
for i in range(10):
    plt.subplot(1,10,i+1)
    plt.imshow(x_train[i])
    plt.title(class_name[y_train[i][0]])
    plt.axis('off')
plt.show()
```



```
In [33]: from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(x_train)
```

```
In [45]: modelA = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
```

```

        layers.Dense(32, activation='softmax')
    ])
    modelA.compile(optimizer='adam',
                   loss='sparse_categorical_crossentropy',
                   metrics=['accuracy'])

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base\_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

In [46]: modelB = models.Sequential([
    layers.Conv2D(32,(3,3), activation='relu', input_shape=(32,32,3)),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(32,(3,3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(32, activation='softmax')
])

modelB.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

```

```

In [47]: from re import VERBOSE

historyA = modelA.fit(datagen.flow(x_train, y_train, batch_size=64), epochs=5,
                      historyB = modelB.fit(datagen.flow(x_train, y_train, batch_size=64), epochs=5

```

Epoch 1/5

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data\_adapters/py\_dataset\_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().\_\_init\_\_(\*\*kwargs)` in its constructor. `\*\*kwargs` can include `workers`, `use\_multiprocessing`, `max\_queue\_size`. Do not pass these arguments to `fit()`, as they will be ignored.

```

self._warn_if_super_not_called()

```

782/782 ————— 92s 115ms/step - accuracy: 0.3012 - loss: 1.9345 -  
 val\_accuracy: 0.5225 - val\_loss: 1.3498  
 Epoch 2/5

782/782 ————— 93s 119ms/step - accuracy: 0.4989 - loss: 1.3986 -  
 val\_accuracy: 0.5974 - val\_loss: 1.1487  
 Epoch 3/5

782/782 ————— 98s 126ms/step - accuracy: 0.5558 - loss: 1.2540 -  
 val\_accuracy: 0.6151 - val\_loss: 1.1059  
 Epoch 4/5

782/782 ————— 95s 121ms/step - accuracy: 0.5886 - loss: 1.1619 -  
 val\_accuracy: 0.6410 - val\_loss: 1.0374  
 Epoch 5/5

782/782 ————— 96s 123ms/step - accuracy: 0.6118 - loss: 1.1044 -  
 val\_accuracy: 0.6425 - val\_loss: 1.0200  
 Epoch 1/5

782/782 ————— 108s 134ms/step - accuracy: 0.3329 - loss: 2.3890  
 - val\_accuracy: 0.5579 - val\_loss: 1.2968  
 Epoch 2/5

782/782 ————— 100s 127ms/step - accuracy: 0.4882 - loss: 1.4499  
 - val\_accuracy: 0.5837 - val\_loss: 1.1924  
 Epoch 3/5

782/782 ————— 100s 128ms/step - accuracy: 0.5351 - loss: 1.3345  
 - val\_accuracy: 0.5743 - val\_loss: 1.2157  
 Epoch 4/5

782/782 ————— 100s 127ms/step - accuracy: 0.5503 - loss: 1.2677  
 - val\_accuracy: 0.4655 - val\_loss: 1.6410  
 Epoch 5/5

782/782 ————— 101s 129ms/step - accuracy: 0.5816 - loss: 1.1876  
 - val\_accuracy: 0.5782 - val\_loss: 1.1650

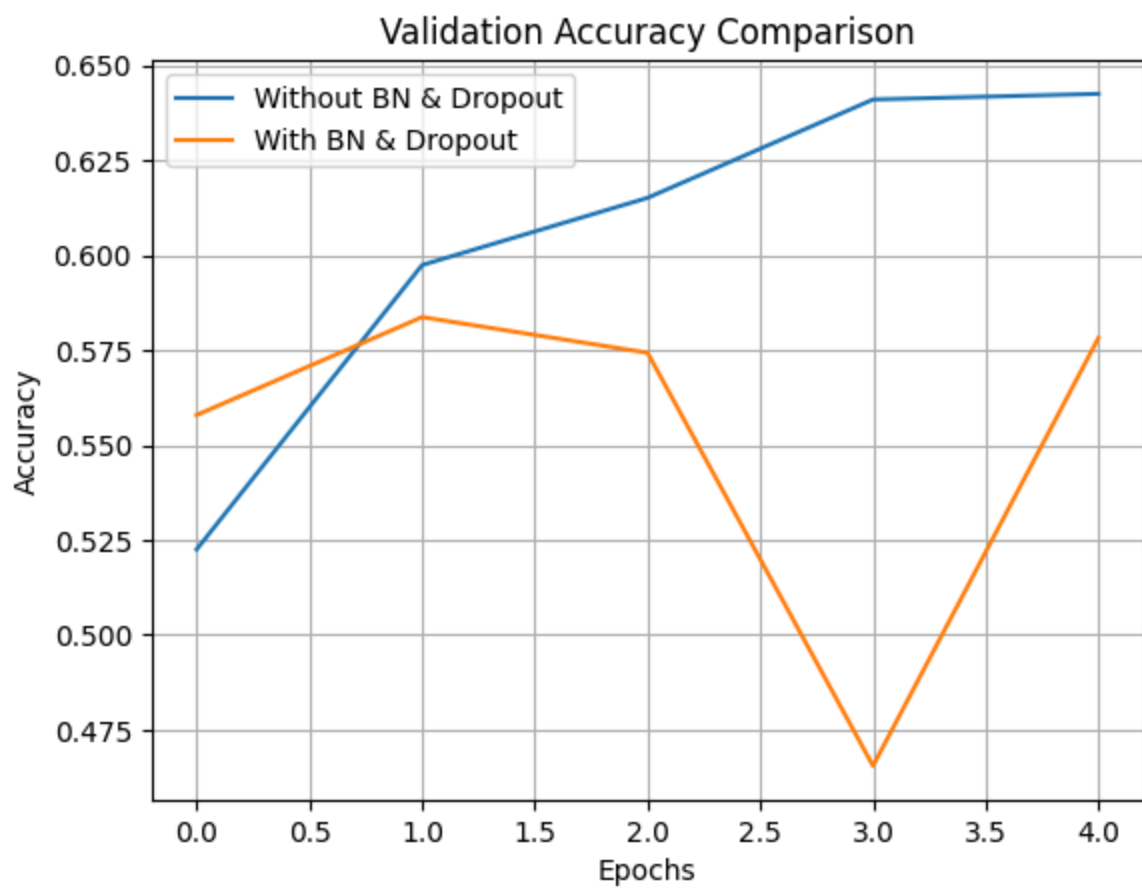
In [ ]:

```
In [53]: test_lossA, test_accA = modelA.evaluate(x_test, y_test, verbose=0)
test_lossB, test_accB = modelB.evaluate(x_test, y_test, verbose=0)
print("\nModel A (Without BN & Dropout) Accuracy:", round(test_accA * 100, 2),
print("Model B (With BN & Dropout) Accuracy:", round(test_accB * 100, 2), "%")
```

Model A (Without BN & Dropout) Accuracy: 64.25 %

Model B (With BN & Dropout) Accuracy: 57.82 %

```
In [54]: plt.plot(historyA.history['val_accuracy'], label='Without BN & Dropout')
plt.plot(historyB.history['val_accuracy'], label='With BN & Dropout')
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



In [ ]: